

IMPLEMENTAÇÃO DE MÁQUINAS ABSTRATAS: AUTÔMATO FINITO DETERMINÍSTICO, AUTÔMATO COM PILHA E MÁQUINA DE TURING

1. INTRODUÇÃO

A teoria da computação constitui um dos pilares fundamentais da ciência da computação, fornecendo o embasamento matemático para a compreensão dos limites do que pode e do que não pode ser computado. Dentro desse campo, o estudo das máquinas abstratas ocupa papel central, pois permite classificar linguagens formais de acordo com sua complexidade e com os recursos computacionais necessários para reconhecê-las ou gerá-las (SIPSER, 2012).

A Hierarquia de Chomsky organiza as linguagens formais em quatro classes distintas: linguagens regulares (Tipo 3), linguagens livres de contexto (Tipo 2), linguagens sensíveis ao contexto (Tipo 1) e linguagens recursivamente enumeráveis (Tipo 0). Cada classe está associada a um modelo computacional específico: os autômatos finitos reconhecem linguagens regulares; os autômatos com pilha reconhecem linguagens livres de contexto; e as máquinas de Turing, modelo mais poderoso, reconhecem as linguagens recursivamente enumeráveis e computam quaisquer funções computáveis (HOPCROFT; MOTWANI; ULLMAN, 2006).

O presente trabalho tem por objetivo implementar, em linguagem C# com a plataforma .NET 10.0, três modelos de máquinas abstratas que exemplificam distintos níveis dessa hierarquia: um Autômato Finito Determinístico (AFD) para reconhecer a linguagem $L1 = \{w \in \{a,b\}^* \mid w \text{ termina com o sufixo "ab"}\}$; um Autômato com Pilha (AP) não determinístico para reconhecer $L2 = \{a^n b^n \mid n \geq 1\}$ e $L3$ (palíndromos sobre $\{a,b\}^*$ com comprimento ≥ 1); e uma Máquina de Turing (MT) determinística para reconhecer $L4 = \{a^n b^n c^n \mid n \geq 1\}$ e computar a função unária $f(n) = n+1$. A partir dessas implementações, são discutidas as decisões de projeto, os resultados dos testes e as diferenças de poder computacional entre os três modelos.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Autômato Finito Determinístico

Um Autômato Finito Determinístico é formalmente definido como uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde Q é um conjunto finito de estados, Σ é o alfabeto de entrada, $\delta: Q \times \Sigma \rightarrow Q$ é a função de transição total, $q_0 \in Q$ é o estado inicial e $F \subseteq Q$ é o conjunto de estados de aceitação (SIPSER,

2012). A cada símbolo lido da entrada, o autômato realiza exatamente uma transição, tornando o processo estritamente determinístico.

Para a linguagem L1, o AFD implementado possui:

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = q_0$$

$$F = \{q_2\}$$

A função de transição δ é descrita pela tabela a seguir:

Estado	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

($\rightarrow$ indica estado inicial; $*$ indica estado de aceitação)

Intuição: q_0 representa o estado em que nenhum progresso foi feito em direção ao sufixo "ab"; q_1 indica que o último símbolo lido foi "a"; q_2 indica que os dois últimos símbolos foram "ab".

Exemplo de cálculo manual para "bab":

$$\delta(q_0, b) = q_0$$

$$\delta(q_0, a) = q_1$$

$$\delta(q_1, b) = q_2 \rightarrow q_2 \in F \rightarrow \text{ACEITA}$$

Exemplo de cálculo manual para "ba":

$$\delta(q_0, b) = q_0$$

$$\delta(q_0, a) = q_1 \rightarrow q_1 \notin F \rightarrow \text{REJEITA}$$

2.2 Autômato com Pilha

Um Autômato com Pilha é formalmente definido como uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, onde Q é o conjunto de estados, Σ é o alfabeto de entrada, Γ é o alfabeto da pilha, $\delta: Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ é a função de transição não determinística, q_0 é o estado inicial, $Z_0 \in \Gamma$ é o símbolo inicial da pilha e $F \subseteq Q$ é o conjunto de estados de aceitação (HOPCROFT; MOTWANI; ULLMAN, 2006). O não determinismo é capturado pelo contradomínio ser um conjunto de pares (estado, string de pilha), e as λ -transições permitem mudanças de estado sem consumir símbolos da entrada.

A implementação adota aceitação por pilha vazia: uma cadeia é aceita quando toda a entrada foi consumida e a pilha está vazia.

Para a linguagem $L_2 = \{a^n b^n \mid n \geq 1\}$:

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{A, Z\}$$

$$Z_0 = Z$$

$$F = \{\} \text{ (aceitação por pilha vazia)}$$

Transições:

$$\delta(q_0, a, Z) = \{(q_0, AZ)\} \quad \text{— empilha A sobre Z ao ler primeiro 'a'}$$

$$\delta(q_0, a, A) = \{(q_0, AA)\} \quad \text{— empilha A ao ler 'a' subsequente}$$

$$\delta(q_0, \lambda, Z) = \{(q_1, \lambda)\} \quad \text{— transição } \lambda \text{ para fase de desempilhamento}$$

$$\delta(q_0, b, A) = \{(q_1, \lambda)\} \quad \text{— desempilha A ao ler 'b'}$$

$$\delta(q_1, b, A) = \{(q_1, \lambda)\} \quad \text{— continua desempilhando}$$

$$\delta(q_1, \lambda, Z) = \{(q_2, \lambda)\} \quad \text{— esvazia Z da pilha ao final}$$

Exemplo de cálculo manual para "aabb":

$$(q_0, aabb, Z) \rightarrow (q_0, abb, AZ) \rightarrow (q_0, bb, AAZ)$$

$$\rightarrow (q_1, b, AZ) \rightarrow (q_1, \epsilon, Z) \rightarrow (q_2, \epsilon, \epsilon) \rightarrow \text{pilha vazia, entrada consumida} \rightarrow$$

ACEITA

Para a linguagem L_3 (palíndromos sobre $\{a,b\}^*$), o AP emprega não determinismo para decidir, em cada posição da entrada, se está na primeira metade (fase de empilhamento) ou na segunda metade (fase de desempilhamento). O estado intermediário representa o "centro" do palíndromo.

Exemplo de cálculo manual para "aba":

$$(q_0, aba, Z) \rightarrow \text{empilha 'a': } (q_0, ba, aZ)$$

— não determinismo: tenta transição λ para q_1 (centro)
 $(q_1, ba, aZ) \rightarrow$ lê 'b' sem consumir pilha (palíndromo ímpar: centro é descartado)
 $\rightarrow (q_1, a, aZ) \rightarrow$ desempilha 'a': $(q_2, \varepsilon, Z) \rightarrow$ esvazia $Z \rightarrow$ ACEITA

2.3 Máquina de Turing

Uma Máquina de Turing determinística é formalmente definida como uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{aceitar}, q_{rejeitar})$, onde Q é o conjunto de estados, Σ é o alfabeto de entrada (não contém o símbolo branco '_'), $\Gamma \supseteq \Sigma$ é o alfabeto da fita (contém '_'), $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição parcial, q_0 é o estado inicial, $q_{aceitar}$ é o estado de aceitação e $q_{rejeitar}$ é o estado de rejeição (SIPSER, 2012). A fita é infinita em ambas as direções; a cabeça de leitura/escrita pode mover-se para a esquerda (L) ou para a direita (R).

Para a linguagem $L_4 = \{a^n b^n c^n \mid n \geq 1\}$:

$Q = \{q_0, q_1, q_2, q_3, q_4, q_{accept}, q_{reject}\}$

$\Sigma = \{a, b, c\}$

$\Gamma = \{a, b, c, A, B, C, _\}$

O algoritmo opera por marcação iterativa:

1. q_0 : Varre à direita; ao encontrar 'a' não marcado, substitui por 'A' e vai a q_1 ; se encontrar 'B' (todos os 'a' foram marcados), vai a q_4 .
2. q_1 : Varre à direita sobre 'a' e 'B' já marcados; ao encontrar 'b' não marcado, substitui por 'B' e vai a q_2 .
3. q_2 : Varre à direita sobre 'b' e 'C' já marcados; ao encontrar 'c' não marcado, substitui por 'C' e vai a q_3 .
4. q_3 : Varre à esquerda até encontrar branco '_' (início da fita), retorna a q_0 .
5. q_4 : Verifica que restam apenas B, C e branco; se sim, vai a q_{accept} .

Exemplo de cálculo manual para "abc" (passos simplificados):

q_0 : lê 'a' $\rightarrow$ escreve 'A', move R $\rightarrow q_1$

q_1 : lê 'b' $\rightarrow$ escreve 'B', move R $\rightarrow q_2$

q_2 : lê 'c' $\rightarrow$ escreve 'C', move L $\rightarrow q_3$

q_3 : varre L até branco $\rightarrow$ move R $\rightarrow q_0$

q_0 : lê 'B' $\rightarrow$ vai a q_4

q_4 : lê 'B', 'C', '_' $\rightarrow$ vai a $q_{accept} \rightarrow$ ACEITA

Para a função unária $f(n) = n+1$ (sucessor unário), a MT possui apenas 2 transições:

$\delta(q_0, 1) = (q_0, 1, R)$ — varre todos os '1's para a direita

$\delta(q_0, _) = (q_{\text{accept}}, 1, R)$ — ao encontrar branco, escreve '1' e aceita

3. DESCRIÇÃO DA IMPLEMENTAÇÃO

3.1 Estrutura Geral dos Projetos

Os três projetos seguem uma arquitetura comum: carregamento de configuração via arquivo JSON (com fallback para configuração embutida em código), exibição do diagrama/tabela da máquina, leitura de cadeias de um arquivo de entradas e execução com rastreamento passo a passo. Cada projeto é uma aplicação console independente em C# (.NET 10.0), organizada nos arquivos de configuração, carregador, máquina e ponto de entrada.

3.2 Autômato Finito Determinístico (Parte1)

A função de transição δ é representada por um Dictionary<(string estado, char simbolo), string>, que associa cada par (estado atual, símbolo lido) ao estado destino. Essa estrutura foi escolhida por oferecer acesso em $O(1)$ e mapear diretamente a definição formal da função de transição total.

O carregamento da configuração é feito pela classe AFDLoader, que desserializa o arquivo afd.json para o tipo AFDConfig utilizando System.Text.Json. O objeto AFDConfig armazena as listas de estados, alfabeto, estado inicial, estados de aceitação e as transições como registros TransicaoConfig. Caso o arquivo não seja encontrado, o Program.cs instancia o AFD com a configuração padrão embutida.

O método Aceitar(string cadeia) percorre cada símbolo da entrada, aplica a função de transição e registra o caminho percorrido, exibindo ao final o traço de execução no formato " $q_0 \rightarrow q_1 \rightarrow q_2$ ". Se algum símbolo não pertencer ao alfabeto, a execução é interrompida com rejeição imediata. O método ExibirDiagrama() renderiza a tabela de transições com marcadores visuais para estado inicial ($\rightarrow$) e estados de aceitação (*).

3.3 Autômato com Pilha (Parte2)

A função de transição δ não determinística é representada por um Dictionary<(string estado, char entrada, char topoFilha), List<(string novoEstado, string empilhar)>>, onde cada chave mapeia para uma lista de possíveis transições. O símbolo λ é representado pelo caractere '\0', permitindo que λ -transições coexistam naturalmente na mesma estrutura.

O núcleo da simulação está no método privado Simular(), que implementa busca recursiva por retrocesso (backtracking). A cada passo, o método tenta todas as transições aplicáveis ao estado atual, símbolo de entrada e topo da pilha, incluindo as λ -transições. A condição de aceitação é verificada quando toda a entrada foi consumida e a pilha está vazia. Qualquer ramo que não leve à aceitação é abandonado e o retrocesso prossegue para o próximo ramo disponível.

A pilha é gerenciada como Stack<char>. O método ClonarPilhaEPush() cria uma cópia independente da pilha e aplica o empilhamento da string resultante da transição, garantindo que cada ramo de exploração mantenha seu próprio estado de pilha. O rastreamento da execução exibe, a cada passo, a tripla (estado, entrada restante, conteúdo da pilha).

O Program.cs instancia dois autômatos distintos: um configurado para L2 e outro para L3, cada um com seu respectivo arquivo de entradas. As transições são adicionadas manualmente via chamadas a AdicionarTransicao() no construtor de cada instância.

3.4 Máquina de Turing (Parte3)

A fita infinita é representada por um Dictionary<int, char>, onde as chaves são posições inteiras e os valores são os símbolos armazenados. Posições ausentes no dicionário são lidas como branco '_'. Essa representação permite que a fita cresça em qualquer direção sem alocação prévia de memória, armazenando apenas as células não em branco.

A função de transição δ é representada por um Dictionary<(string estado, char simbolo), (string novoEstado, char novoSimbolo, char direcao)>, mapeando cada par (estado, símbolo lido) para a tripla de ação. O carregamento é feito por MTLoader, que desserializa os arquivos mt_l4.json (com 46 transições e 7 estados) e mt_unario.json (com 2 transições e 3 estados).

O método Aceitar(string cadeia) implementa o loop de simulação: inicializa a fita com a cadeia de entrada nas posições 0..n-1, executa até atingir qaccept, qreject, ou o limite de 10.000 passos, e a cada iteração lê o símbolo sob o cabeçote, busca a transição correspondente, escreve o novo símbolo, move o cabeçote e muda de estado. Transições não definidas causam rejeição implícita.

O método ExibirPasso() exibe o número do passo, o estado atual, o conteúdo da fita e a posição do cabeçote (marcado entre colchetes, por exemplo: "A [b] c").

O Program.cs carrega e executa as duas máquinas sequencialmente, lendo os arquivos entradas_l4.txt e entradas_unario.txt. Para a máquina unária, o resultado exibido inclui o conteúdo final da fita após a aceitação.

4. RESULTADOS E TESTES

Todos os casos de teste foram executados com sucesso, produzindo os resultados esperados conforme a definição formal de cada linguagem. As tabelas a seguir apresentam os resultados obtidos.

Tabela 1 – Resultados para L1 (AFD)

#	Cadeia	Esperado	Resultado
1	ab	Aceita	Aceita
2	aab	Aceita	Aceita
3	bab	Aceita	Aceita
4	a	Rejeita	Rejeita
5	b	Rejeita	Rejeita
6	ba	Rejeita	Rejeita
7	ababab	Aceita	Aceita
8	aaabbb	Rejeita	Rejeita
9	(vazia)	Rejeita	Rejeita
10	aabb	Rejeita	Rejeita

Fonte: elaborado pelos autores.

Tabela 2 – Resultados para L2 (AP)

#	Cadeia	Esperado	Resultado
1	ab	Aceita	Aceita

2	aabb	Aceita	Aceita
3	aaabbb	Aceita	Aceita
4	aab	Rejeita	Rejeita
5	abb	Rejeita	Rejeita
6	ba	Rejeita	Rejeita
7	(vazia)	Rejeita	Rejeita
8	abab	Rejeita	Rejeita

Fonte: elaborado pelos autores.

Tabela 3 – Resultados para L3 – Palíndromos (AP)

#	Cadeia	Esperado	Resultado
1	ab	Aceita	Aceita
2	aba	Aceita	Aceita
3	abba	Aceita	Aceita
4	ab	Rejeita	Rejeita
5	aab	Rejeita	Rejeita
6	(vazia)	Rejeita	Rejeita

Fonte: elaborado pelos autores.

Tabela 4 – Resultados para L4 (MT)

#	Cadeia	Esperado	Resultado
1	abc	Aceita	Aceita
2	aabbcc	Aceita	Aceita
3	aaabbbccc	Aceita	Aceita
4	aabbc	Rejeita	Rejeita
5	ab	Rejeita	Rejeita
6	abcabc	Rejeita	Rejeita

7	(vazia)	Rejeita	Rejeita
---	---------	---------	---------

Fonte: elaborado pelos autores.

Tabela 5 – Resultados para $f(n) = n+1$ (MT Unária)

#	Entrada	Saída Esperada	Resultado
1	1	11	Aceita
2	111	1111	Aceita
3	11111	111111	Aceita
4	(vazia)	1	Aceita

Fonte: elaborado pelos autores.

5. ANÁLISE COMPARATIVA

As três máquinas implementadas representam níveis distintos da Hierarquia de Chomsky, evidenciando uma relação de inclusão estrita: toda linguagem reconhecida por um AFD é reconhecida por um AP, e toda linguagem reconhecida por um AP é reconhecida por uma MT; as recíprocas, entretanto, não valem (LEWIS; PAPADIMITRIOU, 1997).

O AFD opera sem memória auxiliar, além dos estados de controle finito. Sua principal virtude é a eficiência: o reconhecimento de qualquer cadeia de comprimento n exige exatamente n passos e espaço constante. Em contrapartida, o poder expressivo do AFD é limitado às linguagens regulares — classes como $L_2 = \{a^n b^n\}$ não podem ser reconhecidas por nenhum AFD, pois a necessidade de contar e comparar quantidades arbitrárias ultrapassa a capacidade de um controle finito de estados (SIPSER, 2012). O Lema do Bombeamento para linguagens regulares formaliza essa limitação.

O AP adiciona uma pilha de memória ilimitada, o que lhe permite reconhecer todas as linguagens livres de contexto, incluindo L_2 e L_3 . O não determinismo é um recurso essencial para o reconhecimento de palíndromos (L_3), pois é necessário "adivinhar" o centro da cadeia sem conhecer seu comprimento a priori. Contudo, a pilha é uma estrutura LIFO que permite acesso apenas ao topo, impossibilitando o controle simultâneo de três contadores independentes. Por isso, $L_4 = \{a^n b^n c^n\}$

não é livre de contexto e, portanto, está além do alcance de qualquer AP (HOPCROFT; MOTWANI; ULLMAN, 2006).

A MT é o modelo mais poderoso: com acesso irrestrito a uma fita de trabalho ilimitada e a capacidade de mover o cabeçote em ambas as direções, ela reconhece todas as linguagens recursivamente enumeráveis e computa todas as funções computáveis (incluindo $f(n) = n+1$). O algoritmo de marcação iterativa para L4 utiliza a fita para simular três contadores simultaneamente, o que seria impossível para um AP.

A tabela a seguir resume as capacidades de cada modelo:

Linguagem/Função	AFD	AP	MT
L1: termina com "ab"	Sim	Sim	Sim
L2: $a^n b^n$	Não	Sim	Sim
L3: palíndromos	Não	Sim	Sim
L4: $a^n b^n c^n$	Não	Não	Sim
$f(n) = n+1$ (unário)	Não	Não	Sim

Fonte: elaborado pelos autores.

6. CONCLUSÃO

O desenvolvimento deste trabalho proporcionou uma experiência concreta de como conceitos teóricos da computação se traduzem em estruturas de software. A implementação do AFD reforçou a simplicidade e eficiência dos autômatos finitos, ao mesmo tempo em que tornou evidente sua incapacidade de lidar com dependências de contagem. A construção do AP foi o maior desafio técnico do projeto: modelar o não determinismo por meio de backtracking recursivo exigiu cuidado especial no gerenciamento de cópias independentes da pilha para cada ramo de execução, evitando interferências entre os caminhos explorados. Por fim, a Máquina de Turing demonstrou como a generalização da fita de memória é suficiente para superar todas as limitações anteriores — a representação da fita como um dicionário de inteiros para caracteres revelou-se uma solução elegante para simular a infinitude teórica com recursos finitos de memória.

A Hierarquia de Chomsky deixou de ser apenas um resultado teórico abstrato para tornar-se uma ferramenta de raciocínio direta: ao deparar-se com uma linguagem como L4, o conhecimento de que ela exige uma MT — e não um AFD ou AP — orienta imediatamente as decisões de projeto. Essa capacidade de classificar problemas pelo poder computacional necessário para resolvê-los é, em última análise, a contribuição mais valiosa que a teoria da computação oferece à prática da engenharia de software.

REFERÊNCIAS

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introduction to Automata Theory, Languages, and Computation. 3. ed. Boston: Pearson, 2006.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. Elements of the Theory of Computation. 2. ed. Upper Saddle River: Prentice Hall, 1997.

SIPSER, Michael. Introduction to the Theory of Computation. 3. ed. Stamford: Cengage Learning, 2012.